SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	S Susmitha	**Reg. No.:**	192571068
Section / Batch:	Slot B	**Date:**	19.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Code of Conduct

I S Susmitha (192571068) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 1

Name	Register Number	Course Code	Course
S Susmitha	192571068	CSA0603	Design and Analysis of Algorithms

Question :

Given two strings, determine the shortest string that contains both as subsequences. The objective is to minimize the length of the resulting string while preserving the order of characters. Efficient merging of subsequences is required. Use Dynamic Programming to compute the solution.

Sample Input:

s1 = AGGTAB

s2 = GXTXAYB

Expected Output:

Length = 9

SHORTEST COMMON SUPERSEQUENCE

Aim

To design and implement an efficient algorithm, using Dynamic Programming, that computes the Shortest Common Supersequence (SCS) of two given strings s1 and s2 — that is, the shortest possible string in which both s1 and s2 occur as subsequences — and to analyze its time and space complexity.

Algorithm

SCS_DP(s1, s2):

1. Let m = length(s1) and n = length(s2). Create a 2-D table dp of size (m+1) × (n+1), where dp[i][j] will store the length of the Longest Common Subsequence (LCS) of the prefixes s1[0..i-1] and s2[0..j-1].

2. Initialize the base cases: dp[i][0] = 0 for all i, and dp[0][j] = 0 for all j, since the LCS of any string with an empty string is 0.

3. Fill the table using the recurrence, for i = 1..m and j = 1..n:

if s1[i-1] == s2[j-1]: dp[i][j] = dp[i-1][j-1] + 1

else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])

4. The length of the LCS of s1 and s2 is now dp[m][n]. By the identity length(SCS) = m + n − length(LCS), the minimum possible length of the supersequence is determined without yet building the string.

5. Reconstruct the SCS by tracing back through the table starting at cell (m, n):

- If s1[i-1] == s2[j-1]: this character is common to both strings — append it once and move diagonally to (i-1, j-1).
- Else if dp[i-1][j] > dp[i][j-1]: the character s1[i-1] is not part of the LCS — append it and move to (i-1, j).
- Else: the character s2[j-1] is not part of the LCS — append it and move to (i, j-1).

6. When the traversal reaches a row or column boundary (i = 0 or j = 0), append all remaining characters of the non-exhausted string directly, since they cannot be merged further.

7. Reverse the accumulated string (it was built back-to-front) to obtain the final Shortest Common Supersequence.

8. Validate correctness by confirming that both s1 and s2 individually appear as subsequences of the constructed SCS.

Complexity: Time complexity is O(m × n) for filling the DP table plus O(m + n) for reconstruction, giving overall O(m × n). Space complexity is O(m × n) for the table (reducible to O(min(m, n)) if only the length, and not the string, were required).

Program

```
#include <iostream>
#include <vector>
#include <string>
#include <chrono>
#include <iomanip>
#include <cassert>
#include <algorithm>

using namespace std;
using namespace std::chrono;

class SCSSolver {
private:
    string s1, s2;
    int m, n;
    vector<vector<int>> dp;   // dp[i][j] = length of LCS of s1[0..i-1], s2[0..j-1]
    string scsResult;
    long long cellsComputed = 0;
```

```
// Step 1: Build the LCS DP table -- core recurrence of the algorithm
void buildLCSTable() {
    dp.assign(m + 1, vector<int>(n + 1, 0));
    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            } else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
            cellsComputed++;
        }
    }
}
```

```cpp
// Step 2: Reconstruct SCS by walking the DP table from (m, n) back to (0, 0)
string reconstructSCS() {
    int i = m, j = n;
    string result = "";

    while (i > 0 && j > 0) {
        if (s1[i - 1] == s2[j - 1]) {
            // Matching character belongs to the LCS -> appears once in SCS
            result.push_back(s1[i - 1]);
            i--; j--;
        } else if (dp[i - 1][j] > dp[i][j - 1]) {
            // Character unique to s1 must be included
            result.push_back(s1[i - 1]);
            i--;
        } else {
            // Character unique to s2 must be included
            result.push_back(s2[j - 1]);
            j--;
        }
    }
```

```cpp
    // Append any remaining characters (only one of the loops below runs)
    while (i > 0) { result.push_back(s1[i - 1]); i--; }
    while (j > 0) { result.push_back(s2[j - 1]); j--; }

    reverse(result.begin(), result.end());
    return result;
}

// Step 3: Validate that a given string is a subsequence of the target
bool isSubsequence(const string &sub, const string &target) {
    size_t p = 0;
    for (char c : target) {
        if (p < sub.size() && sub[p] == c) p++;
    }
    return p == sub.size();
}
```

```
public:
    SCSSolver(const string &a, const string &b) : s1(a), s2(b), m(a.size()), n(b.size()) {}

    void solve() {
        buildLCSTable();
        scsResult = reconstructSCS();
    }

    int getLCSLength() const { return dp[m][n]; }
    int getSCSLength() const { return (int)scsResult.size(); }
    string getSCS() const { return scsResult; }
    long long getCellsComputed() const { return cellsComputed; }

    bool validate() {
        return isSubsequence(s1, scsResult) && isSubsequence(s2, scsResult);
    }
```

```
    void printDPTable() const {
        cout << "\n      ";
        cout << setw(3) << " ";
        for (int j = 0; j < n; j++) cout << setw(3) << s2[j];
        cout << "\n    " << setw(3) << " ";
        for (int j = 0; j <= n; j++) cout << setw(3) << 0;
        cout << "\n";
        for (int i = 1; i <= m; i++) {
            cout << " " << s1[i - 1] << " ";
            for (int j = 0; j <= n; j++) cout << setw(3) << dp[i][j];
            cout << "\n";
        }
    }
};
```

```
// ------------------------------------------------------------------------
// Helper: runs one test case end-to-end and prints a formatted report
// ------------------------------------------------------------------------
void runTestCase(const string &s1, const string &s2, int caseNumber) {
    cout << "========================================================================\n";
    cout << "TEST CASE " << caseNumber << "\n";
    cout << "========================================================================\n";
    cout << "String 1 (s1) : " << s1 << "   (length = " << s1.size() << ")\n";
    cout << "String 2 (s2) : " << s2 << "   (length = " << s2.size() << ")\n";

    SCSSolver solver(s1, s2);

    auto start = high_resolution_clock::now();
    solver.solve();
    auto end = high_resolution_clock::now();
    double timeMs = duration<double, milli>(end - start).count();

    cout << "\nLCS Table (DP grid):";
    solver.printDPTable();
```

```
    cout << "\nLength of LCS(s1, s2)        : " << solver.getLCSLength() << "\n";
    cout << "Shortest Common Supersequence : " << solver.getSCS() << "\n";
    cout << "Length = " << solver.getSCSLength() << "\n";

    bool valid = solver.validate();
    cout << "Validation (both strings are subsequences of SCS) : "
         << (valid ? "PASSED" : "FAILED") << "\n";
    assert(valid && "SCS validation failed!");

    cout << "\nComplexity Metrics:\n";
    cout << "   DP cells computed : " << solver.getCellsComputed()
         << "  (Time complexity O(m*n))\n";
    cout << "   Space used        : O(m*n) = "
         << (s1.size() + 1) * (s2.size() + 1) << " integers\n";
    cout << "   Execution time    : " << fixed << setprecision(4) << timeMs << " ms\n\n";
}
```

```cpp
int main() {
    // Primary sample from the problem statement
    runTestCase("AGGTAB", "GXTXAYB", 1);

    // Additional test cases to demonstrate robustness of the implementation
    runTestCase("ABCBDAB", "BDCABA", 2);
    runTestCase("GEEK", "EQG", 3);
    runTestCase("", "ABC", 4);        // edge case: one empty string
    runTestCase("AAAA", "AA", 5);     // edge case: repeated characters

    cout << "======================================================================\n";
    cout << "All test cases completed and validated successfully.\n";
    cout << "======================================================================\n";
    return 0;
}
```

Input

As specified in the problem statement:

s1 = "AGGTAB"
s2 = "GXTXAYB"

Output

```
PS C:\Users\Susmitha\Downloads\CSA0603 Coding Challenge> .\scs.exe
======================================================================
TEST CASE 1
======================================================================
String 1 (s1) : AGGTAB   (length = 6)
String 2 (s2) : GXTXAYB   (length = 7)

LCS Table (DP grid):
          G  X  T  X  A  Y  B
       0  0  0  0  0  0  0  0
 A     0  0  0  0  0  1  1  1
 G     0  1  1  1  1  1  1  1
 G     0  1  1  1  1  1  1  1
 T     0  1  1  2  2  2  2  2
 A     0  1  1  2  2  3  3  3
 B     0  1  1  2  2  3  3  4

Length of LCS(s1, s2)        : 4
Shortest Common Supersequence : AGGXTXAYB
Length = 9
Validation (both strings are subsequences of SCS) : PASSED

Complexity Metrics:
   DP cells computed : 56  (Time complexity O(m*n))
   Space used        : O(m*n) = 56 integers
   Execution time    : 0.0334 ms

======================================================================
TEST CASE 2
======================================================================
String 1 (s1) : ABCBDAB   (length = 7)
String 2 (s2) : BDCABA   (length = 6)
```

```
LCS Table (DP grid):
          B  D  C  A  B  A
        0  0  0  0  0  0  0
 A   0  0  0  0  1  1  1
 B   0  1  1  1  1  2  2
 C   0  1  1  2  2  2  2
 B   0  1  1  2  2  3  3
 D   0  1  2  2  2  3  3
 A   0  1  2  2  3  3  4
 B   0  1  2  2  3  4  4

Length of LCS(s1, s2)        : 4
Shortest Common Supersequence : ABCBDCABA
Length = 9
Validation (both strings are subsequences of SCS) : PASSED

Complexity Metrics:
   DP cells computed : 56  (Time complexity O(m*n))
   Space used        : O(m*n) = 56 integers
   Execution time    : 0.0205 ms

======================================================================
TEST CASE 3
======================================================================
String 1 (s1) : GEEK   (length = 4)
String 2 (s2) : EQG   (length = 3)

LCS Table (DP grid):
          E  Q  G
        0  0  0  0
 G   0  0  0  1
 E   0  1  1  1
 E   0  1  1  1
 K   0  1  1  1

Length of LCS(s1, s2)        : 1
Shortest Common Supersequence : GEEKQG
Length = 6
Validation (both strings are subsequences of SCS) : PASSED
```

```
Complexity Metrics:
   DP cells computed : 20  (Time complexity O(m*n))
   Space used        : O(m*n) = 20 integers
   Execution time    : 0.0134 ms

======================================================================
TEST CASE 4
======================================================================
String 1 (s1) :    (length = 0)
String 2 (s2) : ABC   (length = 3)

LCS Table (DP grid):
          A  B  C
        0  0  0  0

Length of LCS(s1, s2)        : 0
Shortest Common Supersequence : ABC
Length = 3
Validation (both strings are subsequences of SCS) : PASSED

Complexity Metrics:
   DP cells computed : 4  (Time complexity O(m*n))
   Space used        : O(m*n) = 4 integers
   Execution time    : 0.0060 ms

======================================================================
TEST CASE 5
======================================================================
String 1 (s1) : AAAA   (length = 4)
String 2 (s2) : AA   (length = 2)

LCS Table (DP grid):
          A  A
        0  0  0
 A      0  1  1
 A      0  1  2
 A      0  1  2
 A      0  1  2
```

```
Length of LCS(s1, s2)        : 2
Shortest Common Supersequence : AAAA
Length = 4
Validation (both strings are subsequences of SCS) : PASSED

Complexity Metrics:
   DP cells computed : 15  (Time complexity O(m*n))
   Space used        : O(m*n) = 15 integers
   Execution time    : 0.0124 ms

======================================================================
All test cases completed and validated successfully.
======================================================================
```

Result

The Shortest Common Supersequence of s1 = "AGGTAB" and s2 = "GXTXAYB" was successfully computed using the Dynamic Programming approach as "AGGXTXAYB", with a minimum length of 9 — matching the expected output. The Longest Common Subsequence length (4, corresponding to "GTAB") correctly satisfies the identity length(SCS) = len(s1) + len(s2) − len(LCS) = 6 + 7 − 4 = 9. The result was verified programmatically by confirming both input strings occur as subsequences of the generated supersequence, and the algorithm was further validated against four additional test cases, including boundary conditions, all of which passed. The program achieves the optimal O(m × n) time and space complexity for this problem.